



Azurduy, María Inés (mmaria.azurduy@gmail.com)

Clementi, Yamila (yamilaclementi@gmail.com)

Maldonado, Marcos Guido (maldonadomarcos31@gmail.com)

*Programación con objetos II: "Documentación sobre el trabajo práctico integrador
"UNQ-Shop"*

PATRONES DE DISEÑO UTILIZADOS

2.1 Catálogo de productos

Para la creación de este módulo, utilizamos el patrón estructural **Composite**. Dado que permite que el cliente trate elementos simples (producto) y complejos (paquetes), de la misma manera, y poder utilizar la recursión con los paquetes.

Detalles de implementación:

Añadimos un objeto **Atributo**, para que sea escalable en un futuro, pensando en que se puedan añadir nuevos atributos.

Añadimos un objeto **Categoría**, para minimizar errores humanos al momento de la creación de un producto. (Por ejemplo, que un producto tenga una categoría con un error de ortografía).

Roles:

- **Cliente:** Sistema
- **Componente:** Item
- **Hoja:** Producto
- **Compuesto:** Paquete

2.2 Ciclo de vida del pedido

Para la creación de este módulo, utilizamos el patrón de comportamiento **State**. Creamos así, los 6 estados que puede tener un pedido. Cada estado es tratado como un objeto, y tiene una interfaz en común (**IEstado**). El pedido inicia con el estado Borrador.

Detalles de implementación:

Decidimos que la **NotaDeCredito** quede registrada en el pedido y no en el sistema, dado que en otras partes del trabajo no se requería dicha información.

Roles:

- **Contexto:** Pedido
- **Estado:** IEstado
- **Estados Concretos:**
Borrador/Confirmado/EnPreparación/Enviado/Entregado/Cancelado

¿Qué alternativas frágiles evitamos?

La utilización de if-else anidados.

2.3 Métodos de envío

Para la creación de este módulo, utilizamos el patrón **Strategy**. En caso de que surja una nueva forma de envío, simplemente debe añadirse una nueva estrategia concreta, sin tener que modificar la clase contexto.

Roles:

- **Contexto:** Pedido
- **Estrategia:** IFormaDeEnvio
- **Estrategias concretas:** RetiroEnSucursal/EnvioEstandar/EnvioExpress

Detalles de implementación:

La interfaz IFormaDeEnvio declara los métodos en común entre las 3 formas de envío: el cálculo del valor del envío, y la estimación de los días de entrega del pedido.

Creamos un Usuario, ya que lo necesitábamos para la interfaz ICorreoArgentino (que requiere una dirección). Este usuario más adelante nos sería de utilidad, en el (módulo 2.5)

2.4 Métodos de pago

Para la creación de este módulo, utilizamos el patrón **Template Method**. Tal como lo menciona el enunciado, el “procesamiento de un pago siempre sigue los mismos pasos en el mismo orden... Sin embargo, cada medio de pago implementa esos pasos de manera diferente”.

Dentro de la clase abstracta, se crea el método plantilla (que es final), y un método hook para las notificaciones.

Roles:

- **Clase abstracta:** Método de pago
- **Clases concretas:** TarjetaDeCredito/TransferenciaBancaria/BilleteraVirtual

Cada método de pago tiene una API propia del método de pago, que se encarga de hacer las verificaciones correspondientes.

2.5 Notificaciones del Pedido

Para la creación de este módulo, utilizamos el patrón **Observer**. Los subsistemas concretos reaccionan ante los cambios de estado del Pedido, llevando a cabo diferentes acciones. El pedido no conoce a sus suscriptores. Este trabajo es delegado al Notificador. En caso de que se añadan nuevos suscriptores, simplemente deben ser agregados a la lista de suscriptores. El Notificador o sujeto concreto en el pedido que implementa el protocolo notificador de la interfaz I

Roles:

- **Notificador:** INotificador
- **Notificador concreto:** Pedido
- **Interfaz suscriptora:** IObservador
- **Suscriptores concretos:** NotificadorDeEmail /GeneradorDeFactura /Fidelizacion

Detalles de implementación:

Creamos una interfaz "IMailSender" para el envío de emails, tiene 2 metodos para enviar mail uno con direccion de email, titulo y mensaje y otro con los mismos atributos pero agregando un **IAdjunto** (que puede ser un cupón de descuento o una factura)

2.6 Búsqueda en el catálogo

Para la búsqueda de ítems en Catálogo se utilizó el patrón **Composite**, donde el component puede ser tanto un criterio de búsqueda simple (leaf) como compuesto (composite), así al realizar la búsqueda desde el sistema uno solo utiliza un criterio para filtrar los ítems del catálogo.

Detalles de implementación:

Todos los criterios utilizan una interfaz en común llamada ICriterio con una operación de filtro sobre ítems de catálogo que describe la lista de ítems filtrada.

Criterios de búsqueda simples:

- **CriterioPorNombre:** el nombre del ítem contiene el texto ingresado (sin distinción de mayúsculas).
- **CriterioPorPrecioMax:** el precio base del ítem es menor o igual al valor indicado.
- **Criterio Porcategoría:** el ítem pertenece a una categoría dada (electrónica, indumentaria, hogar, etc.).
- **CriterioPorDisponibilidad:** el ítem tiene stock disponible en al menos un depósito.

Criterios de búsqueda complejos:

Los criterios compuestos AND y OR están compuestos de 2 criterios de búsqueda que pueden ser tanto simples, como complejos:

- **CriterioAND:** los ítems de catálogo que satisfacen ambos criterios.
- **CriterioOR:** los ítems de catálogo de ambos criterios.
- **CriterioNOT:** los ítems de catálogo que no satisfacen a ese criterio.

Detalles de implementación:

Los criterios simples (o tipo hoja) no permiten la adición ni eliminación de otros criterios, ya que estas operaciones son exclusivas de los criterios complejos. Cualquier intento de agregar o quitar elementos en un criterio hoja lanzará siempre una excepción.

Roles:

- **Component:** ICriterio.
- **Leaf:** CriterioPorNombre, CriterioPorPrecioMax, CriterioPorCategoria, CriterioPorDisponibilidad.
- **Composite:** CriterioAND, CriterioOR, CriterioNOT.

2.7 Reportes

Para la realización de este último módulo, aplicamos el patrón de comportamiento “Visitor”.

Detalles de implementación:

Creamos una venta, que el sistema registra y guarda en una lista. Esto fue con la intención de que, al generar el reporte, poder filtrar bien por las fechas de inicio y fin, decidiendo no agregarle una fecha al pedido y mejor separar estos datos en la clase Venta.

Creamos también una interfaz IReporte, pensando que podrían haber más tipos de reportes (tal como se mostraba al inicio del trabajo).

Hicimos que el pedido tenga al Sistema como atributo, para al querer registrar una venta, invoque al método propio del sistema para guardar la venta.

Roles:

- **Visitantes concretos:** TextoPlanoExportVisitor, CSVExportVisitor, HTMLExportVisitor
- **Interfaz visitante:** IVisitor
- **Interfaz elemento:** IReporte
- **Elemento concreto:** ReporteProductosMasVendidos
- **EstructuraDeObjetos:** List<Venta>